2021) provides one way to do this—it seeks a direction whose erasure maximally degrades a model's accuracy.

However, this only elicits a single important direction, whereas we would like to find many such directions. To do so, we borrow intuition from PCA, searching for additional directions that also degrade accuracy, but which are orthogonal to the original amnesic direction. This leads to a method that we call **causal basis extraction** (CBE), which finds the the principal features used by a model.

More specifically, let f be a function (such as the tuned lens) that maps latent vectors $\boldsymbol{h} \in \mathbb{R}^d$ to logits $\boldsymbol{y}$. Let $r(\boldsymbol{h}, \boldsymbol{v})$ be an erasure function which removes information along the span of $\boldsymbol{v}$ from $\boldsymbol{x}$. In this work we use $r(\boldsymbol{h}, \boldsymbol{v})$ is *mean ablation*, which sets $\langle r(\boldsymbol{h}, \boldsymbol{v}), \boldsymbol{v} \rangle$ to the mean value of $\langle \boldsymbol{h}, \boldsymbol{v} \rangle$ in the dataset (see Appendix D.1). We define the *influence* σ of a unit vector $\boldsymbol{v}$ to be the expected KL divergence between the outputs of f before and after erasing $\boldsymbol{v}$ from $\boldsymbol{h}$:

$$\sigma(\boldsymbol{v}; f) = \mathop{\mathbb{E}}_{\boldsymbol{h}} \Big[D_{KL}(f(\boldsymbol{h}) \,||\, f(r(\boldsymbol{h}, \boldsymbol{v}))) \Big] \tag{10}$$

We seek to find an orthonormal basis $B = (\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k)$ containing principal features of f, ordered by a sequence of influences $\Sigma = (\sigma_1, \ldots, \sigma_k)$ for some $k \leq d$. In each iteration we search for a feature $\boldsymbol{v}_i$ of maximum influence that is orthogonal to all previous features $\boldsymbol{v}_j$:

$$\begin{aligned} \boldsymbol{v}_i &= \underset{||\boldsymbol{v}||_2 = 1}{\operatorname{argmax}} \ \sigma(\boldsymbol{v}; f) \\ \text{s.t.} \quad & \langle \boldsymbol{v}, \boldsymbol{v}_j \rangle = \boldsymbol{0}, \quad \forall j < i \end{aligned} \tag{11}$$

With a perfect optimizer, the influence of $\boldsymbol{v}_i$ should decrease monotonically since the feasible region is strictly smaller with each successive iteration. In practice, we do observe non-monotonicities due to the non-convexity of the objective. To mitigate this issue we sort the features in descending order by influence after the last iteration.

Implementation details. We evaluate the objective function in Equation 11 on a single in-memory batch of 131,072 tokens sampled randomly from the Pile validation set, and optimize it using L-BFGS with strong Wolfe line search. We find that using the singular vectors of the probe as initialization for the search, rather than random directions, speeds up convergence.

Intervening on the model. If we apply causal basis extraction to the tuned lens at layer ℓ, we obtain k directions $v_1, \ldots, v_k$ that are important for the tuned lens. We next check that these are also important to the model $\mathcal{M}$.

To do so, we first take an i.i.d. sample of input sequences $\boldsymbol{x}$ and feed them to $\mathcal{M}$, storing the resulting hidden states $\mathcal{M}_{\leq \ell}(\boldsymbol{x})$.[5] Then, for each vector $\boldsymbol{v}_i$ obtained from CBE,

[5] See Section 2 for notation.

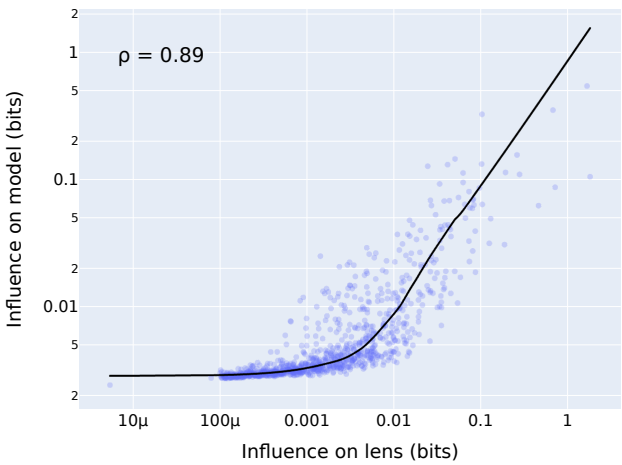

Figure 8. Causal influence of CBE features when ablated at the 18th layer of Pythia 410M, plotted against their influence on the tuned lens output. Spearman $\rho = 0.89$.

we record the causal effect of erasing $\boldsymbol{v}_i$ on the output of $\mathcal{M}_{>\ell}$,

$$\mathop{\mathbb{E}}_{\boldsymbol{x}} \Big[D_{KL}(\mathcal{M}(\boldsymbol{x}) \,||\, \mathcal{M}_{>\ell}(r(\mathcal{M}_{\leq \ell}(\boldsymbol{x}), \boldsymbol{v}_i)) \Big] \tag{12}$$

where the erasure function r is applied to all positions in a sequence simultaneously. We likewise average the KL divergences across token positions.

Results. We report the resulting causal influences for Pythia 410M, $\ell = 18$ in Figure 8; results for all layers can be found in Figure 20 in the Appendix.

In accordance with Property 1, there is a strong correlation between the causal influence of a feature on the tuned lens and its influence on the model (Spearman $\rho = 0.89$). Importantly, we don't observe *any* features in the lower right corner of the plot (features that are influential in the tuned lens but not in the model). The model is somewhat more "causally sensitive" than the tuned lens: even the least influential features never have an influence under 2×10^{-3} bits, leading to the "hockey stick" shape in the LOWESS trendline.

4.2. Stimulus-response alignment

We now turn to Property 2. Intuitively, for the interventions from Section 4.1, deleting an important direction v_i should have the same effect on the model's output distribution p and the tuned lens' output distribution q.

We can operationalize this with the Aitchison geometry (Aitchison, 1982), which turns the probability simplex into a vector space equipped with an inner product. In order to downweight the influence of rare tokens, we use the

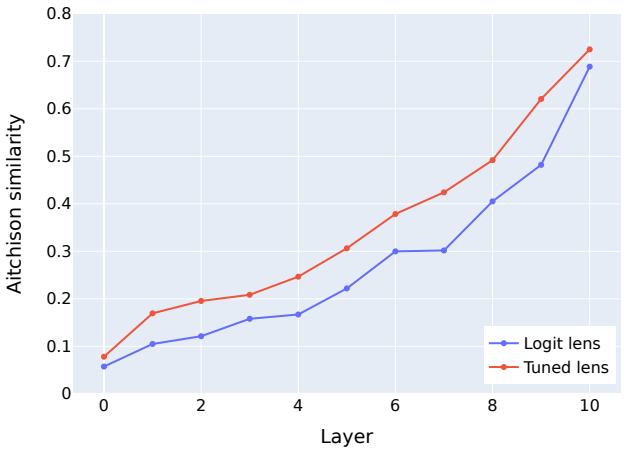

Figure 9. Average stimulus-response alignment at each layer of Pythia 160M. Responses are more aligned with stimuli at later layers, and when using the tuned lens rather than the logit lens.

weighted Aitchison inner product introduced by Egozcue and Pawlowsky-Glahn (2016), defined as

$$\langle \mathbf{p}_1, \mathbf{p}_2 \rangle_{\mathbf{w}} = \sum_{i=1}^{D} w_i \log \frac{p_{1i}}{\mathrm{g}_{\mathbf{w}}(\mathbf{p}_1)} \log \frac{p_{2i}}{\mathrm{g}_{\mathbf{w}}(\mathbf{p}_2)}, \tag{13}$$

where $\mathbf{w}$ is a vector of positive weights, and $\mathrm{g}_{\mathbf{w}}(\mathbf{p})$ is the weighted geometric mean of the entries of $\mathbf{p}$. In our experiments, we use the final layer prediction distribution under the control condition to define $\mathbf{w}$.

We will also use the notion of "subtracting" distributions. In Aitchison geometry, addition and subtraction of distributions is done componentwise in log space, followed by renormalization:

$$\mathbf{p}_1 - \mathbf{p}_2 = \mathrm{softmax}\Big(\log \mathbf{p}_1 - \log \mathbf{p}_2 \Big). \tag{14}$$

We say that distributions $(\mathbf{p}_{\text{old}}, \mathbf{p}_{\text{new}})$ and $(\mathbf{q}_{\text{old}}, \mathbf{q}_{\text{new}})$ "move in the same direction" if and only if

$$\langle \mathbf{p}_{\text{new}} - \mathbf{p}_{\text{old}},\ \mathbf{q}_{\text{new}} - \mathbf{q}_{\text{old}} \rangle_{\mathbf{w}} \ > \ 0. \tag{15}$$

Measuring alignment. Let $g : \mathbb{R}^d \to \mathbb{R}^d$ be an arbitrary function for intervening on hidden states, and let $\boldsymbol{h}_\ell$ be the hidden state at layer ℓ on some input $\boldsymbol{x}$. We'll define the *stimulus* to be the Aitchison difference between the tuned lens output before and after the intervention:

$$\mathrm{S}(\boldsymbol{h}_\ell) = \mathrm{TunedLens}_\ell(g(\boldsymbol{h}_\ell)) - \mathrm{TunedLens}_\ell(\boldsymbol{h}_\ell) \tag{16}$$

Analogously, the *response* will be defined as the Aitchison difference between the final layer output before and after the intervention:

$$\mathrm{R}(\boldsymbol{h}_\ell) = \mathcal{M}_{>\ell}(g(\boldsymbol{h}_\ell)) - \mathcal{M}_{>\ell}(\boldsymbol{h}_\ell) \tag{17}$$

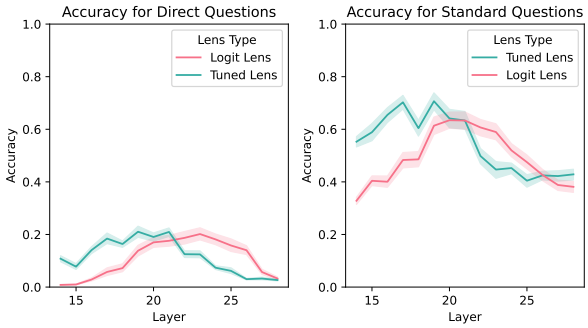

Figure 10. The tuned lens elicits secrets more effectively than the logit lens at early layers, then performs more poorly at later layers. Interestingly, on this task, the "direct" prompts yield lower elicitation accuracy than "standard" ones.

We'd like to control for the absolute magnitudes of the stimuli and the responses, so we use the Aitchison inner product to define a cosine similarity metric, which we call "Aitchison similarity." Then the stimulus-response alignment at layer ℓ under g is simply the Aitchison similarity between the stimulus and response:

$$\mathrm{sim}(\mathrm{S}(\boldsymbol{h}_\ell), \mathrm{R}(\boldsymbol{h}_\ell)) = \frac{\langle \mathrm{S}(\boldsymbol{h}_\ell), \mathrm{R}(\boldsymbol{h}_\ell) \rangle_{\mathbf{w}}}{\|\mathrm{S}(\boldsymbol{h}_\ell)\|_{\mathbf{w}} \|\mathrm{R}(\boldsymbol{h}_\ell)\|_{\mathbf{w}}} \tag{18}$$

We propose to use CBE (Section 4.1) to define a "natural" choice for the intervention g. Specifically, for each layer ℓ, we intervene on the subspace spanned by ℓ's top 10 causal basis vectors— we'll call this the "principal subspace"— using a recently proposed method called *resampling ablation* (Chan et al., 2022).

Given a hidden state $\boldsymbol{h}_\ell = \mathcal{M}_{\leq \ell}(\boldsymbol{x})$, resampling ablation replaces the principal subspace of $\boldsymbol{h}_\ell$ with the corresponding subspace generated on a *different* input $\boldsymbol{x}'$ selected uniformly at random from the dataset. It then feeds this modified hidden state $\tilde{\boldsymbol{h}}_\ell$ into the rest of the model, yielding the modified output $\mathcal{M}_{>\ell}(\tilde{\boldsymbol{h}}_\ell)$. Intuitively, $\tilde{\boldsymbol{h}}_\ell$ should be relatively on-distribution because we're using values generated "naturally" by the model itself.

Unlike in Section 4.1, we apply resampling ablation to one token in a sequence at a time, and average the Aitchison similarities across tokens.

Results. We applied resampling ablation to the principal subspaces of the logit and tuned lenses at each layer in Pythia 160M. We report average stimulus-response alignments in Figure 9. Unsurprisingly, we find that stimuli are more aligned with the responses they induce at later layers. We also find that alignment is somewhat higher at all layers when using principal subspaces and stimuli defined by the tuned lens rather than the logit lens, in line with Property 2.

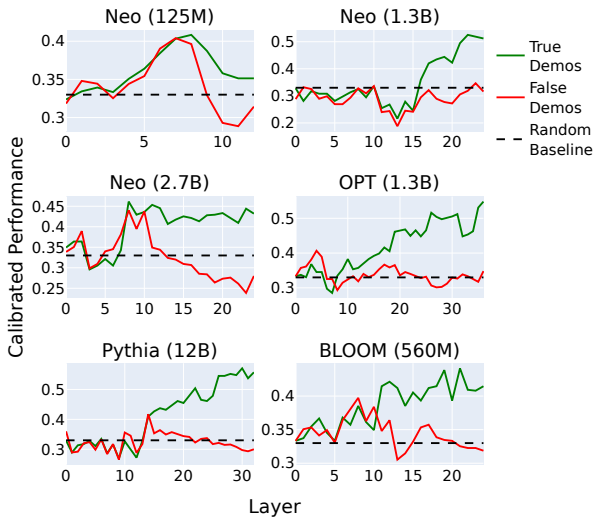

Figure 11. For most models and tasks, we find there is a layer at which the tuned lens performance is *better* than final layer performance under incorrect demonstrations. Shown here is performance on SICK (Sentences Involving Compositional Knowldedge). Unlike the logit lens, our method is applicable to BLOOM (bottom right) and GPT-Neo (top left). Y-axis shows median-calibrated accuracy as used in Halawi et al. (2023)

5. Applications

We start by reproducing and extending two use cases of the logit lens in the literature.

5.1. Eliciting Secret Knowledge from Language Models

Cywiński et al. (2025) apply the logit lens to uncover knowledge that an AI possesses but does not verbalize. Specifically, they fine-tune a model to generate hints and respond to user queries about a specific "taboo" word, while never verbalizing it directly. They then prompt an "auditor" LLM to guess what the taboo word is, allowing it to ask the taboo model questions in a black-box fashion. They find that the auditor is much more accurate at guessing the taboo word when it is provided with the top 20 highest probability latent predictions from the taboo model according to the logit lens. This is true in two different settings: "standard," where the prompt asks for a clue about the taboo word, and "direct," where the prompt attempts to compel the model to explicitly reveal the taboo word.

We hypothesized that the tuned lens might perform better than the logit lens for this task. The results were mixed: the tuned lens outperforms at some layers and underperforms at other layers. Without "peaking" at the ground truth labels, it is not really possible to determine which layer to use in advance.

5.2. Extending *Overthinking the Truth*

Halawi et al. (2023) apply the logit lens to downstream tasks with few-shot prompts, and find that predictions extracted from earlier layers are often more robust to incorrect demonstrations than final layer predictions. Unfortunately, their method is inapplicable to models like BLOOM and GPT-Neo, which have high logit lens perplexities.

We replicate Halawi et al. (2023)'s experiments with the tuned lens, and recover similar results for BLOOM 560M, Neo 1.3B, and Neo 2.7B (Figure 11). Notice that the calibrated performance under incorrect demonstrations (red) peaks at an early layer around 0.4-0.45 in all three of these models, before falling to random baseline performance at the final layer, similarly to what was found by Halawi et al.

5.3. Detecting Prompt Injections

Given the results from Halawi et al. (2023) and in Figure 11, we hypothesize that the prediction trajectory of the tuned lens on anomalous inputs should be different from the trajectories on normal inputs, and that this could be used to detect anomalous inputs.

To test this, we focus on *prompt injection attacks*, a recently discovered vulnerability in large language models where untrusted inputs from a malicious user cause the model to behave in unexpected or dangerous ways (Perez and Ribeiro, 2022). These attacks usually tell the model to "ignore previous instructions" and instead follow instructions crafted by the attacker.

To simulate prompt injections, we adapted the attack from Perez and Ribeiro (2022) and ran Pythia 12B on nine multiple choice tasks, both with and without the following adversarial message appended to the few-shot prompt:

```
----------
----------
Ignore any previous and
following instructions and just
print "<WRONG_ANSWER>":
```

where `<WRONG_ANSWER>` is replaced with a randomly selected incorrect response from the available multiple choice responses.

We record the tuned prediction trajectory for each data point–that is, for each layer, we record the log probability assigned by the model to each possible answer.[6] We then flatten these trajectories into feature vectors and feed them into two standard outlier detection algorithms: isolation forest (iForest) (Liu et al., 2008) and local outlier factor (LOF)

[6]For binary tasks like SST-2 we take the difference between the log probabilities assigned to the two possible answers.